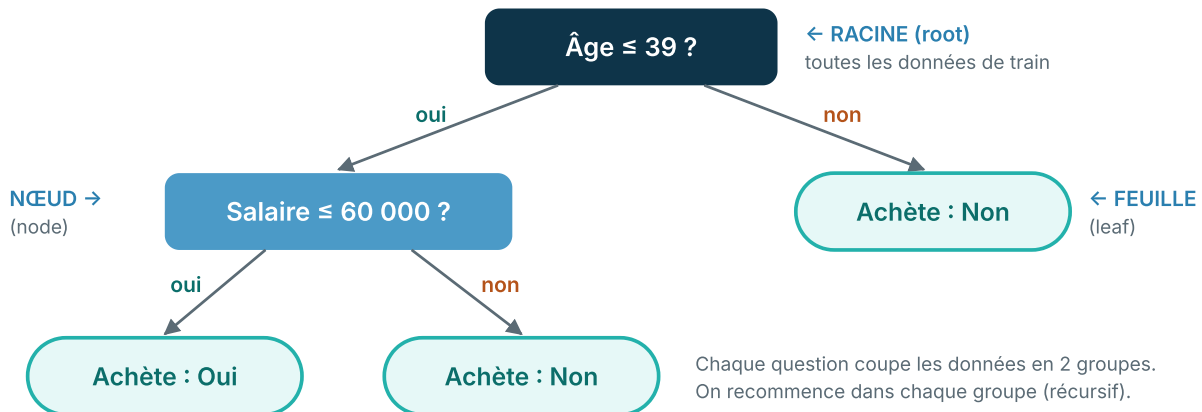


Arbres de décision & Random Forest

Fiche simple · JEDHA Fullstack · M04 Machine Learning — D03 · 2 octobre 2026

En une phrase. Un **arbre de décision**, c'est le jeu des « 20 questions » : on pose des questions oui/non les unes après les autres jusqu'à pouvoir donner une réponse. Une **Random Forest** (forêt aléatoire), c'est demander l'avis de plein d'arbres un peu différents, puis prendre la réponse de la majorité.

1. L'arbre de décision : à quoi ça ressemble



Exemple inventé pour illustrer (variables du dataset du cours : Âge, Salaire → Achète oui/non).

CART (Classification And Regression Trees) : le nom de la famille. Le même principe sert à deux choses : **classification** = prédire une catégorie (oui/non) → la feuille donne une classe ; **régression** = prédire un nombre (un prix) → la feuille donne une valeur (la moyenne du groupe).

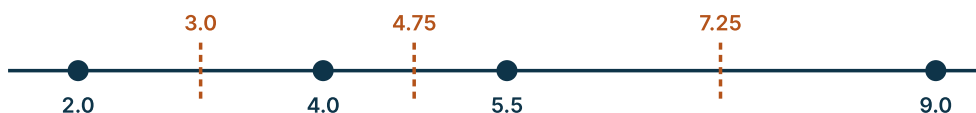
Partition récursive : « couper, puis recouper chaque morceau de la même façon », jusqu'à ce qu'on s'arrête.

2. Comment l'arbre choisit ses questions

À chaque nœud, l'arbre **essaie toutes les questions possibles** (chaque colonne, chaque seuil) et garde celle qui sépare le mieux les données, c'est-à-dire celle qui donne deux groupes les plus « propres » possible.

a) Quels seuils sont testés ?

On trie les valeurs de la colonne et on teste **le milieu entre deux valeurs voisines**.



4 valeurs → 3 seuils candidats (en orange). Règle générale : M valeurs → $M - 1$ seuils possibles.

b) Comment on note une coupe : l'impureté

Un groupe est **pur** si tout le monde dedans a la même réponse. Il est **impur** si les réponses sont mélangées. L'arbre garde la coupe qui donne **le moins d'impureté au total** (gauche + droite).

Impureté / hétérogénéité (H) : une note de « mélange » d'un groupe. $H = 0$ → groupe parfaitement pur. H au maximum → tout est mélangé à parts égales.

Type de problème	Mesure utilisée	En clair
Classification (Y = catégorie)	Indice de Gini $H = \sum p \times (1 - p)$ p = proportion de chaque classe dans le groupe	Mesure le mélange des classes. Tout « Oui » → Gini = 0 (pur). 50 % Oui / 50 % Non → $0,5 \times 0,5 + 0,5 \times 0,5 = \mathbf{0,5}$ (le pire à 2 classes). 75 % / 25 % → $0,75 \times 0,25 \times 2 = \mathbf{0,375}$.
Régression (Y = nombre)	Variance moyenne des écarts au carré par rapport à la moyenne du groupe	Mesure si les valeurs du groupe sont proches les unes des autres. Valeurs toutes identiques → variance = 0. Valeurs très dispersées → grande variance.

Algorithme glouton (greedy) : l'arbre prend la meilleure coupe *tout de suite*, sans réfléchir aux coups suivants. Conséquence : une mauvaise coupe au début se répercute sur tout le reste, et l'arbre final n'est pas forcément le meilleur possible.

Coupe admissible : une coupe valable, c'est-à-dire qui ne laisse aucun des deux groupes vide.

3. Quand l'arbre s'arrête, et le piège de l'overfitting

Si on le laisse faire, l'arbre coupe jusqu'à ce que chaque feuille soit pure : il finit par **apprendre les données par cœur**, y compris le hasard et les erreurs qu'elles contiennent.

Overfitting (sur-apprentissage) : excellent sur les données d'entraînement, mauvais sur de nouvelles données. Comme un élève qui a appris les corrigés par cœur au lieu de comprendre. C'est **le** défaut des arbres.

Arbre trop petit

Trop peu de questions, réponses grossières.

Biais élevé, variance faible

= underfitting (sous-apprentissage).

Arbre bien réglé

Assez de questions pour capter les vraies règles, pas le bruit.

Le bon compromis.

Arbre trop profond

Une règle par cas particulier.

Biais faible, variance élevée
= overfitting.

Biais : le modèle est trop simple, il se trompe « toujours dans le même sens ». **Variance** : le modèle est trop sensible, il change complètement si on change un peu les données d'entraînement. On cherche un équilibre : c'est le **compromis biais-variance**.

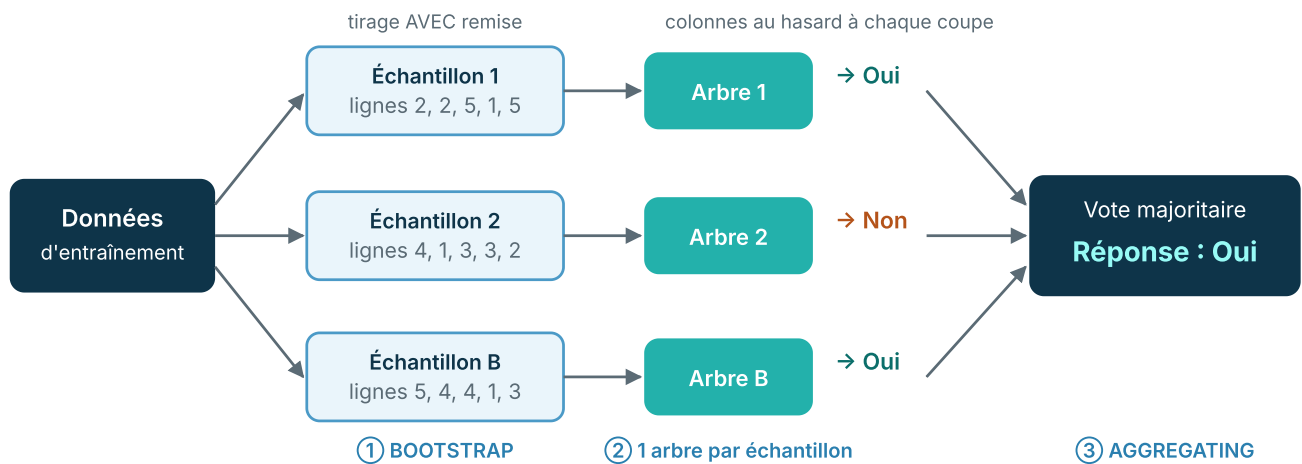
Les freins qu'on peut régler (hyperparamètres)

Hyperparamètre	Ce que ça veut dire	Effet
max_depth	Nombre maximum de questions à la suite (profondeur de l'arbre).	Plus petit → arbre plus simple.
min_samples_split	Nombre minimum de lignes dans un nœud pour avoir le droit de le couper.	Plus grand → moins de coupes.
min_samples_leaf	Nombre minimum de lignes dans chaque feuille.	Plus grand → pas de feuille pour un seul cas isolé.

Hyperparamètre : un réglage que *nous* choisissons avant l'entraînement (le modèle ne l'apprend pas tout seul). **Régularisation** : tout ce qui freine le modèle pour l'empêcher d'apprendre par cœur.

4. La Random Forest : plein d'arbres qui votent

Un arbre seul est instable (forte variance). L'idée : entraîner **beaucoup d'arbres tous un peu différents**, puis combiner leurs réponses. Les erreurs de chacun se compensent, comme quand on demande l'avis de 100 personnes au lieu d'une seule.



Bagging = **Bootstrap** + **Aggregating**. La Random Forest = du bagging d'arbres + un 2^e hasard sur les colonnes.

Bootstrap : fabriquer un nouveau jeu de données en piochant des lignes au hasard **avec remise** (on remet la ligne dans le sac après l'avoir tirée). Une même ligne peut donc sortir plusieurs fois, d'autres pas du tout. Chaque arbre voit ainsi une version légèrement différente des données.

Aggregating (agrégation) : combiner les réponses de tous les arbres. **Classification** → vote majoritaire. **Régression** → moyenne des prédictions.

Méthode d'ensemble : un modèle fait de plusieurs modèles qui travaillent ensemble. La Random Forest en est une.

Les deux sources de hasard (d'où le « Random »)

Hasard n°1 : les lignes	Hasard n°2 : les colonnes
Chaque arbre reçoit son propre échantillon bootstrap.	À chaque coupe , l'arbre n'a le droit de regarder qu' une partie des colonnes , tirée au hasard : environ $\sqrt{p}$ colonnes en classification, $p/3$ en régression (p = nombre total de colonnes).

Pourquoi ce 2^e hasard ? Sans lui, tous les arbres commenceraient par la même colonne « star » et se ressembleraient trop : un vote entre copies conformes n'apporte rien. Avec lui, les arbres sont plus **indépendants**, le calcul par arbre est plus léger, et les colonnes corrélées (qui racontent la même chose) pèsent moins.

Le raisonnement à retenir : chaque arbre de la forêt est laissé très profond → **biais faible mais variance élevée**. En faisant la moyenne de beaucoup d'arbres, **la variance baisse**. Résultat : un modèle précis et stable.

n_estimators : le nombre d'arbres dans la forêt. C'est l'hyperparamètre en plus par rapport à un arbre seul.

5. Feature importance : quelles colonnes comptent le plus ?

Une forêt est moins lisible qu'un arbre seul, mais elle sait dire **quelles colonnes l'ont le plus aidée**.

Méthode	Principe en clair
Mean Decrease in Gini baisse moyenne d'impureté	À chaque coupe, on regarde de combien la colonne utilisée a fait baisser l'impureté. On additionne pour chaque colonne (en tenant compte du nombre de lignes concernées). C'est ce que donne <code>feature_importances_</code> dans <code>scikit-learn</code> .
Mean Decrease in Accuracy baisse moyenne de précision	On mélange au hasard les valeurs d'une colonne et on regarde si le modèle devient moins bon. S'il se dégrade beaucoup → la colonne était importante. S'il ne change pas → elle ne servait à rien.
Importance globale	La moyenne de l'importance de chaque colonne sur tous les arbres de la forêt.

Intérêt par rapport à des scores plus simples : cette importance capte aussi les relations **non linéaires** (qui ne suivent pas une ligne droite) entre une colonne et la cible.

Attention au One-Hot Encoding : une colonne catégorielle découpée en plusieurs colonnes 0/1 (Country → Country_Germany, Country_Spain) voit son importance **diluée** entre ces colonnes. Si besoin, regrouper les catégories rares avant l'entraînement.

6. Ce que ça change dans le code

Point	À retenir
Pas de StandardScaler	Un arbre compare seulement « plus petit / plus grand que le seuil » : l'échelle des colonnes ne change rien. On garde l'imputation des valeurs manquantes (<code>SimpleImputer</code>) et l'encodage des catégories (<code>OneHotEncoder</code>).
Les modèles	<code>DecisionTreeClassifier()</code> et <code>RandomForestClassifier()</code> (versions ... <code>Regressor</code> pour prédire un nombre).
Réglage automatique	<code>GridSearchCV(classifier, param_grid=params, cv=3)</code> teste toutes les combinaisons d'hyperparamètres de la grille et garde la meilleure (<code>best_params_</code> , <code>best_score_</code>).
Probabilités	<code>predict_proba()</code> donne la probabilité de chaque classe. Pour un arbre, c'est la proportion des classes dans la feuille où tombe la ligne (dans la démo : 0,33 / 0,67 = une feuille avec 1 Non et 2 Oui).
Importance	<code>gridsearch.best_estimator_.feature_importances_</code>

Grid Search : essayer systématiquement toutes les combinaisons d'une grille de réglages. **Cross-validation (cv=3)** : couper le train en 3 parts, entraîner sur 2 et tester sur la 3^e, en tournant ; ça donne une note plus fiable qu'un seul essai.

Lecture des résultats de la démo : le dataset n'a que **10 lignes** (8 en train, 2 en test). Les scores ne veulent donc rien dire en eux-mêmes. On y voit quand même l'overfitting : la Random Forest fait **1,0 en train** et **0,5 en test** (elle a appris les 8 lignes par cœur). La démo sert à montrer le *code*, pas la performance.

7. Arbre seul ou forêt : le résumé

	Arbre de décision	Random Forest
Points forts	Très lisible : on peut suivre chaque règle et expliquer une décision. Pas de mise à l'échelle. Supporte bien les valeurs extrêmes (outliers). Choisit seul les colonnes utiles.	Plus précise et plus stable. Beaucoup moins d'overfitting. Donne l'importance des colonnes.
Points faibles	Overfitting facile. Instable. Glouton. En régression, prédit en « marches d'escalier » (paliers) : mauvais pour une relation lisse (droite, courbe).	Moins interprétable (des dizaines d'arbres). Plus lourde en mémoire et en calcul.
Réglages clés	<code>max_depth</code> , <code>min_samples_split</code> , <code>min_samples_leaf</code>	Les mêmes + <code>n_estimators</code>

Les 4 idées à garder en tête. ① Un arbre = une suite de questions oui/non choisies pour rendre les groupes les plus purs possible (Gini ou variance). ② Laissez libre, un arbre apprend par cœur : on le freine avec `max_depth` , `min_samples_split` , `min_samples_leaf` . ③ Random Forest = bagging (bootstrap + vote) + colonnes tirées au hasard à chaque coupe → la variance baisse. ④ Pas besoin de normaliser, et on récupère l'importance des colonnes.